

وحدة متقدمة اختيارية

الأتمتة الاحترافية: CI وحماية الفروع والإصدارات

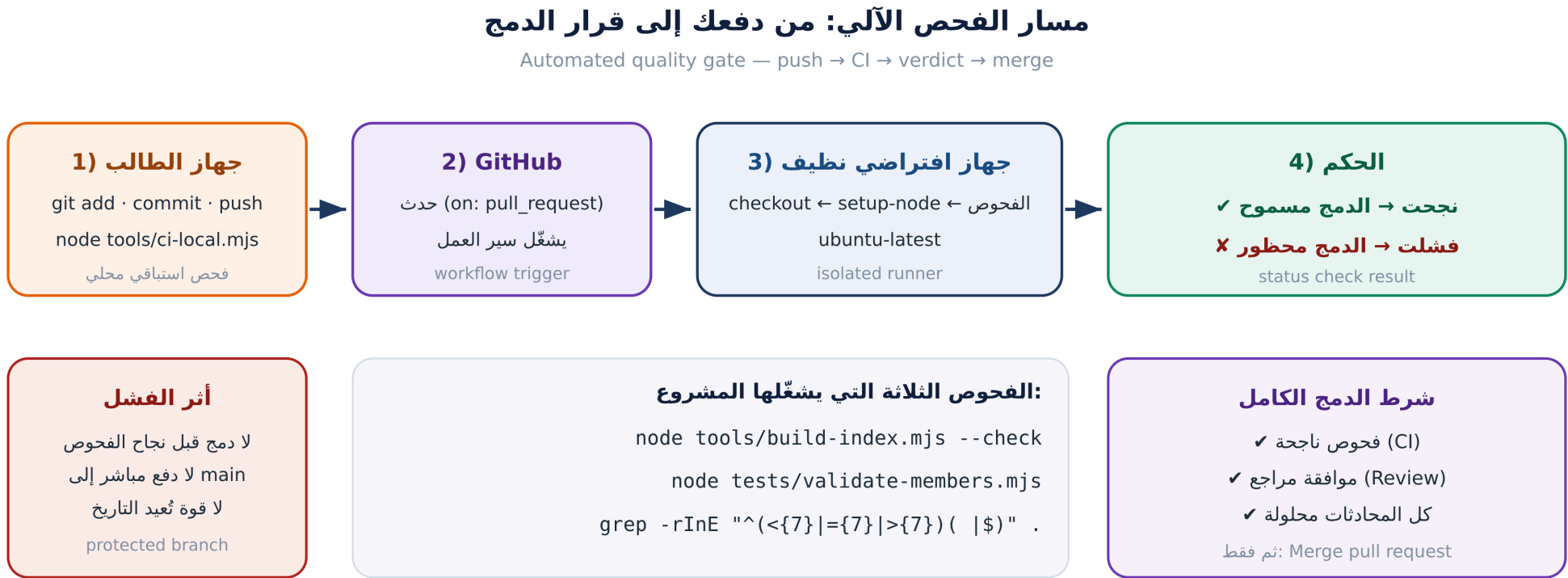
Advanced Unit 06 · Automation, Protection & Releases

- المشروع أصبح يعمل ومنشوراً: نعرفه الآن **يرفض** العمل السيئ تلقائياً
- اليوم نضيف ثلاث قدرات يطلبها كل فريق محترف:
- فحص آلي عند كل طلب سحب (GitHub Actions)
- قواعد تمنع الوقوع في الخطأ (حماية الفرع + CODEOWNERS)
- إصدار رسمي موثّق مع سجل تغييرات (SemVer + CHANGELOG)
- المخرجات: طلب سحب أحمر يُصلح حتى يصبح أخضر، ووسم `v1.0.0` حقيقي

هذه الوحدة اختيارية، لكنها **الأقرب لبيئة العمل الحقيقية** في البرنامج كله.

- أمس: كان المراجع البشري هو خط الدفاع الوحيد
- المراجع يسهو، والوقت قصير، والملفات كثيرة
- الحل: مراجع آلي صبور لا يمل، يفحص كل طلب سحب بنفس المعايير
- لكن الفحص الآلي **لا يغني عن المراجعة البشرية** — بل يحزّرها للتفكير

الفحص الآلي يقدّم **حكماً** (نجاح/فشل)، وحماية الفرع تفرض **قواعد وصول**. كلاهما ضروري، ولا أحدهما بديل عن الآخر.



لا في الذاكرة — والفحص الذي لا يفشل أبداً أسوأ من عدم وجود فحص (Pipeline) القاعدة: الجودة تُبنى في المسار.

```
# .github/workflows/validate.yml
name: validate
on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]
concurrency:
  group: validate-${{ github.ref }}
  cancel-in-progress: true
jobs:
  checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: node tools/build-index.mjs
      - run: node tests/validate-members.mjs
      - run: git diff --exit-code -- data/members-index.json data/members-bundle.js
```

- متى يعمل؟ (عند كل طلب سحب نحو main, وعند كل دفع إلى main)
- أين يعمل؟ (على خادم GitHub في بيئة نظيفة ubuntu-latest, لا على جهازك)
- ما الخطوات الثلاث؟ (توليد الفهرس ← فحص البطاقات ← مطابقة الملفات المولدة)
- لماذا concurrency مع cancel-in-progress ؟ (يلغي الفحص القديم عند دفعة جديدة، فلا ننتظر نتائج قديمة)

سؤال حاسم: ماذا تكشف الخطوة الأخيرة `git diff --exit-code` ؟ ← نسيان إعادة توليد الفهرس بعد إضافة بطاقة — أشهر خطأ في هذا المشروع.

```
$ node tools/ci-local.mjs
===#لمحلي CI فحص محلي مكافئ لـ===
[1/3] توليد الفهرس ..... ✓
[2/3] فحص البطاقات ..... ✓
[3/3] مطابقة الملفات المولدة ..... ✓
--✓ النتيجة: كل الفحوص نجحت ---
```

عادة احترافية: شغّل الفحص محلياً **قبل** الدفع. لا ترسل عملاً ليقوم غيرك باكتشاف أخطائه.

```
# الفخ الأول: بطاقة تحمل وسم CHANGEME
$ node tests/validate-members.mjs
(موجود ولم يُستبدل CHANGEME الوسم) خطأ: يبدو أن البطاقة نسخة غير معدّلة ✕
الملف: data/members/omar-qa.json

# الفخ الثاني: نسيت إعادة توليد الفهرس
$ git diff --exit-code -- data/members-index.json
diff --git a/data/members-index.json b/data/members-index.json
--- a/data/members-index.json
+++ b/data/members-index.json
@@ -5,6 +5,7 @@
   { "username": "leen-docs", ... }
+  { "username": "omar-qa", ... }
exit status 1      الفحص سيفشل حتى تصلح هذا ←
```

- المطلوب: خَرِّب المشروع عن قصد في فرع تجريبي، وشاهد الفحص يفشل، ثم أصلحه

المهمة 3 — شبكة إضافية: التحقق الضعيف يكذب

```
# فحص قديم وحيد:  
$ grep -rInE "^(<<<<<<|>>>>>>|=====)$" --exclude-dir=.git .
```

```
# لماذا يكون صحيحاً أحياناً؟ لأنه  
# (بعدها مسافة ثم كلمة) "<<<<<< HEAD" يفشل في كشف -  
# أنشأها الطلاب بعلامات دمج .txt. يفشل في كشف ملفات -
```

لذلك المهمة الأخيرة في سجل التغييرات تكتب الفحص الصحيح:

```
grep -rInE "^(<{7}|={7}|>{7})($)" --exclude-dir=.git .
```

■ من يكتشف ضعف الفحص القديم ويوثّقه يحصل على 5 درجات إضافية

```
$ node tools/build-index.mjs
[build-index] ✓ بطاقة 7 تم توليد الفهرس:

$ node tests/validate-members.mjs
---بطاقة , 0 خطأ , 0 تحذير 7 / Result: النتيجة ---

$ git add data/members-index.json data/members-bundle.js
$ git commit -m "chore(data): regenerate members index after adding card"
$ git push
```

- راقب تبويب Actions في طلب السحب: **أصفر** (يعمل) ← **أخضر** ✓ (نجح)
- القاعدة: لا تدمج ولا تطلب مراجعة وأنت ترى دائرة صفراء أو حمراء

القاعدة	القيمة	لماذا
منع الدفع المباشر إلى main	✓	لا تجاوز للمراجعة
طلب موافقة واحدة على الأقل	1	مراجعة بشرية حقيقية
فحص مطلوب قبل الدمج	validate	لا دمج بفحص فاشل
منع القوة (force push)	✓	لا حذف لتاريخ الفريق
منع حذف الفرع	✓	لا فقدان مفاجئ

حماية الفرع ليست ثقة بالآخرين، بل تقليل لاحتمال الخطأ البشري — بما فيه خطؤك أنت.

```
# .github/CODEOWNERS
# الترتيب مهم: الأخص سطرًا يفوز
/data/members/site-config.json @instructor-team
/data/members/ @instructor-team
/tests/ @instructor-team @qa-lead
/docs/ @docs-team
* @instructor-team
```

- مالك الكود لا يُستدعى تلقائياً للدمج، بل يُطلب تلقائياً كمراجع — والموافقة شرط
- لاحظ: سطر `/data/members/` يحمي بطاقات الطلاب، لا يسمح بتعديل بيانات عضوية أساسية

```
$ git switch main && git pull --ff-only
$ git tag -a v1.0.0 -m "صفحات النشر + CI + الإصدار الأول: بطاقات الفريق"
Tagger: Workshop Instructor <instructor@example.com>
Date: Tue Oct 6 12:05:41 2026 +0300

$ git show v1.0.0 --stat --no-patch
commit 0b1c2d3... ← يشير إلى عقدة الدمج الرئيسية

$ git push origin v1.0.0
To https://github.com/<org>/class-team-hub.git
 * [new tag] v1.0.0 → v1.0.0
```

`git ls-remote --tags origin` هو **الدليل الوحيد** على أن الوسم على الخادم. وسم محلي لا يظهر لأحد ولا يمكن لأحد استخدامه.

معنون (annotated)	خفيف (lightweight)
git tag -a v1.0.0 -m "..."	git tag v1.0.0
الأمر	من وسم؟
اسم ووقت ورسالة	لا تسجيل
الإصدارات الرسمية	علامات محلية سريعة
الشائع في	الاستخدام مع Releases
الموصى به ✓	غير مناسب

أوامر الفحص: `git ls-remote --tags origin` · `git show v1.0.0 --stat --no-patch` · `git tag -n1` · `git tag -l`

نوع التغيير	مثال من مشروعنا	الرقم الذي يزيد
كسر التوافق	تغيير بنية بطاقة موجودة	MAJOR (2.0.0)
ميزة جديدة متوافقة	صفحة «الأسئلة الشائعة»	MINOR (1.1.0)
إصلاح	تصحيح محاذاة البطاقة	PATCH (1.0.1)

```
$ git tag -l
v1.0.0

$ git tag -n1
v1.0.0
```

CI + الإصدار الأول: بطاقات الفريق

- القاعدة: القيمة (MAJOR) الأكثر خطورة على المستخدمين — لا ترفعها إلا عند كسر توافق

```
$ git add CHANGELOG.md
$ git commit -m "docs(changelog): add release notes for v1.0.0"
$ git push
```

GitHub → Releases → Draft a new release

Tag: v1.0.0
Target: main
Title: أول إصدار مستقر - v1.0.0
Description: CHANGELOG.md ملخص التغييرات من
→ Publish release

```
# من CHANGELOG.md
## [1.0.0] - 2026-10-06
### Added
- بطاقات تعريفية لكل أعضاء الفريق -
- (validate.yml) فحص آلي عند كل طلب سحب -
```

■ الفرق: **الوسم** علامة في التاريخ، **الإصدار** صفحة موثقة بملفات نهائية

- **سؤال الحماية:** ما الفرق بين «فحص مطلوب» و«حماية الفرع»؟
- الجواب: الفحص يعطي حكماً على الجودة؛ الحماية تفرض قواعد الوصول والدمج.
- **سؤال الإصدار:** ما الذي يجعل الإصدار رسمياً؟
- الجواب: وسم معنون مرفوع + ملفات نهائية + سجل تغييرات.

- «أجرتنا تعلّم كلاماً، وهذه الوحدة أظهرت لي ما يحدث فعلاً عند كل دفع.»
- «رأيت أول مرة فحصاً يفشل بسبب ملف لم أولّده — درس لا يُنسى.»
- «صفحة الإصدار على GitHub تجعل مشروعنا يبدو حقيقياً.»

إتقان الأربعة — CI + الحماية + CODEOWNERS + الإصدار — هو الفرق بين «أعرف Git» و«جاهز للعمل في فريق هندسي».

- المشروع الآن: فحص آلي يحمي الجودة، قواعد تحمي الفرع، وإصدار موثّق يحمل رقمه
- هذا ملف أعمال حقيقي يمكنك عرضه في السيرة الذاتية
- طريقتك في الحديث عنه في المقابلة: «بنيت مشروعاً يستخدم Git Flow، مع CI على كل طلب سحب، وحماية فرع main، وإصدار معنون»
- شكراً لكم — البرنامج ينتهي هنا، لكن المستودع يبقى لكم

الخطوة التالية بعد الورشة: طبّق نفس العملية على مشروعك الشخصي، وشارك الرابط معي للمراجعة.